

CprE 3810 Lab 1 Report

Problem 1: Design Cycle

C. Example Inputs/Outputs

1) $iW = 9$, $iLdW = 1$, $iY = 16$, $iX = 4$

$oX = 4$, $oY = 9 \cdot 4 + 16 = 52$

2) $iW = 4$, $iLdW = 1$, $iY = 10$, $iX = 3$

$oX = 3$, $oY = 4 \cdot 3 + 10 = 22$ (My Birthday)

3) $iW = 8$, $iLdW = 1$, $iY = 30$, $iX = 6$

$oX = 6$, $oY = 8 \cdot 6 + 30 = 78$

D. Draw Design

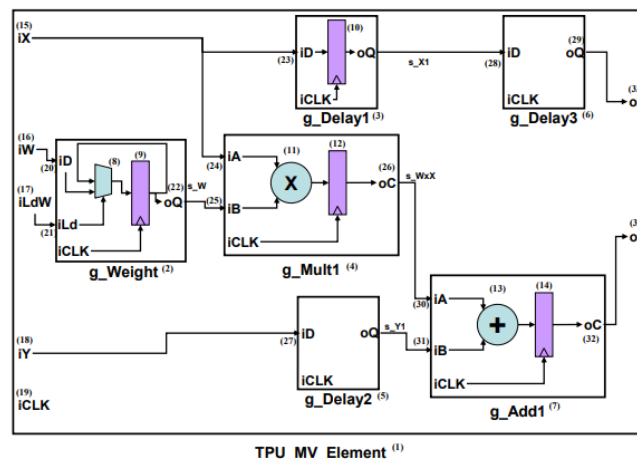


Figure 1: TPU_MV_ELEMENT

Figure 0: Schematic Diagram of TPU to design in VHDL

E. VHDL Design

In the attached diagram, (1) is the TPU_MV_Element name for the entire module. It is defined as an entity in lines 24 through 36 of TPU_MV_Element.vhd. It has four inputs and two outputs. Its structure is defined in the file, complemented by modules from other files.

(7) is the label for the final adder module, g_Add1. It is assigned to the adder module structure from Adder.vhd in line 131 of TPU_MV_Element.vhd as g_Add1: Adder. It has three inputs: s_WxX, s_Y1, &iCLK. It only has one output, oY. These are defined in lines 135 through 138.

(22) is the oQ output of the weight loader module. It is defined in line 30 of RegLD.vhd as a DATA_WIDTH-bit std_logic_vector. Its final assignment is the mapping of the internal storage signal to the final output, oQ, so that the output stays the same if the control signal stays low.

(28) is the output of the first delay module, g_Delay1. The module output, oQ, is mapped to signal s_X1 in line 106 of TPU_MV_Element.

F. VDHL Testbench Modification

```
-- Test case 3:
-- Perform average example of an input activation of 4 and a partial sum of 16. The weight is 9.
s_iX <= x"04"; -- 3 in hexadecimal
s_iW <= x"09"; -- Not strictly necessary, but this makes the testcases easier to read
s_iLdW <= '1'; -- Make sure we continue to load.
s_iY <= x"10"; -- 16 in hexadecimal
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 52 = 9x4+16 and o_X output signal to be 4 after two positive edge of clock.

-- Test case 4:
-- Perform average example of an input activation of 3 and a partial sum of 10. The weight is 4.
s_iX <= x"03"; -- 3 in hexadecimal
s_iW <= x"04"; -- Not strictly necessary, but this makes the testcases easier to read
s_iLdW <= '1'; -- Make sure we continue to load.
s_iY <= x"0A"; -- 10 in hexadecimal
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 22 = 4*3 + 10 and o_X output signal to be 3 after two positive edge of clock.

-- Test case 5:
-- Perform average example of an input activation of 6 and a partial sum of 30. The weight is 8.
s_iX <= x"06"; -- 6 in hexadecimal
s_iW <= x"08"; -- Not strictly necessary, but this makes the testcases easier to read
s_iLdW <= '1'; -- Make sure we continue to load.
s_iY <= x"1E"; -- 30 in hexadecimal
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 78 = 6x8 + 30 and o_X output signal to be 3 after two positive edge of clock.
```

Figure 1: Screenshot from tb_TPU_MV_Element.vhd showing the test case implementations

G. Simulating in QuestaSim

With the files provided, while they all compile correctly, the test bench does not simulate because of errors in the code. The first error is an incorrect declaration of an output as an

input in the multiplier, which wouldn't let the file simulate. The second error is a mapping of an X output to a Y output in the multiplier, which is a functionality error.

H. Debugged TPU Design

After debugging the design with Questasim using the intermediate signals to see where the functionality of the design broke down, I got it to output the following waveforms. As seen from the oY waveform, while it does take three clock cycles to update properly with an incorrect intermediary value, the correct output is computed for each case.

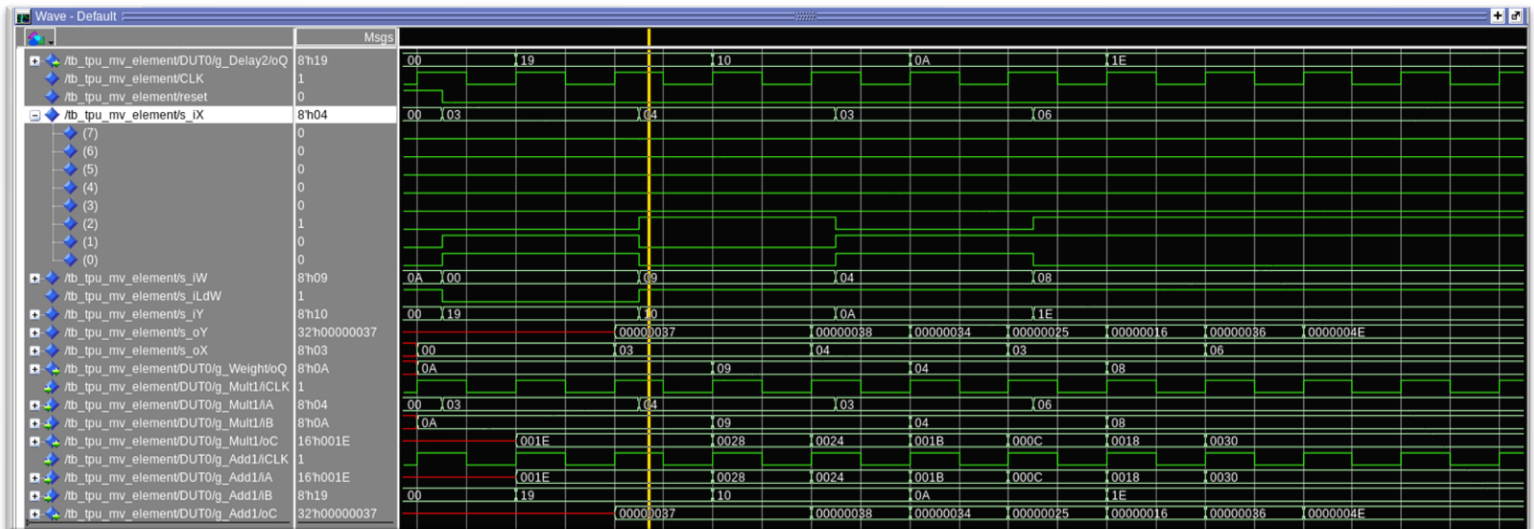


Figure 2: Final debugged TPU_MV_Element testbench waveforms

Problem 3. 2-to-1 MUX design

A. Truth Table, Boolean Equation, & Boolean Circuit

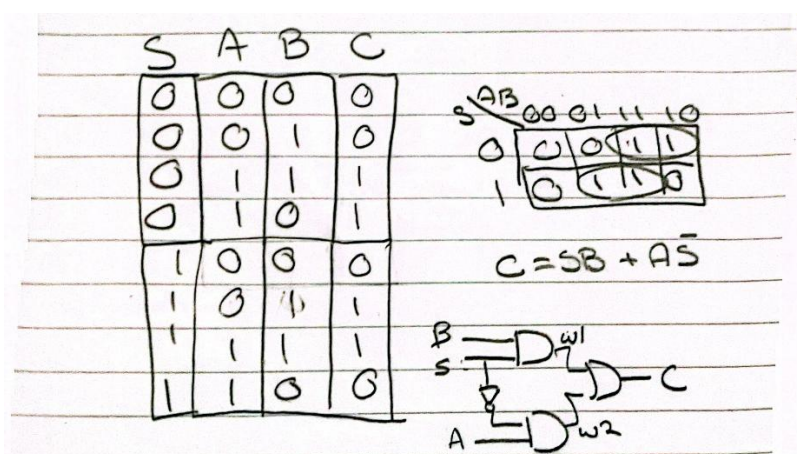


Figure 3: Notebook page with Boolean Components

B. Structural VHDL MUX design

The design for my MUX is saved in mux2t1.vhd within the zip file included

C. VHDL MUX Testbench

The VHDL MUX Testbench is saved in tb_mux2t1.vhd

D. MUX Questasim Output



Figure 4: 2-to1 Mux Testbench output waveform from Questasim

As seen in figure 4, when the select bit, s0, is high, the B input is fed to the output, and vice versa when s0 is low.

E. Dataflow VHDL MUX Design

The model and testbench are saved in mux2t1_df.vhd and tb_mux2t1_df.vhd. The results from the simulation are shown in figure 5.

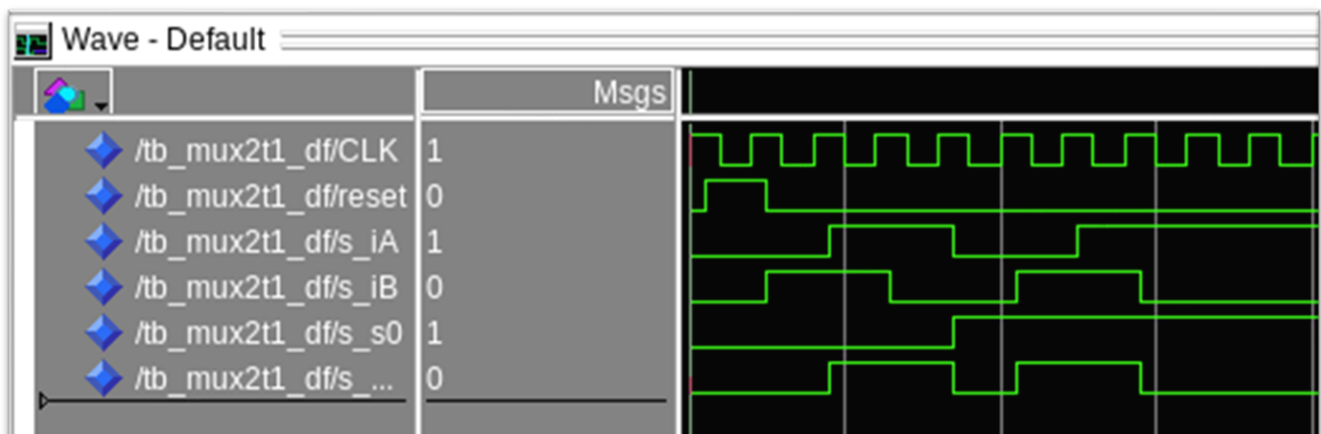


Figure 5: Dataflow Mux simulation output

Problem 4. Using Generics in VHDL

Using generics in VHDL allows for designs with repeating components to be implemented with loops and less lines of code. This part of the lab utilized the previous mux design and a generic statement to create an N-bit input 2-to-1 mux. I made a testbench for this design and the waveform of this testbench is shown below in figure 6.

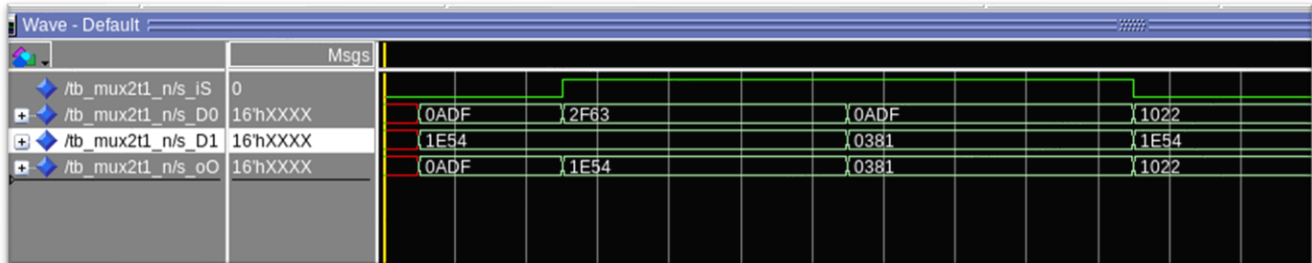


Figure 6: N-bit 2-to-1 Mux Testbench output

Problem 5. One's Complementor

Using the same principle of generics, I made an N-bit complementor with a repeating array on inverters. The design is saved in the file `onescomp.vhd`. Then, I made a testbench for this



design and the output is shown in figure 7.

Figure 7: N-bit Complementor Testbench output

Problem 6. Full Adder and N-bit Adder

A. Full Adder Truth Table, Boolean Algebra, & Schematic

A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A	B	C _{in}	S
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

A	B	C _{in}	C _{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$S = A'BC' + A'B'C + ABC + AB'C'$$
$$= C(A'B \oplus B'A) \oplus AB$$
$$= C(A \oplus B) \oplus AB$$

$$C_{out} = AB + BC + AC = C(B+A) + AB$$
$$= C(A \oplus B) + AB$$

Figure 8: S & Cout logic and individual circuits

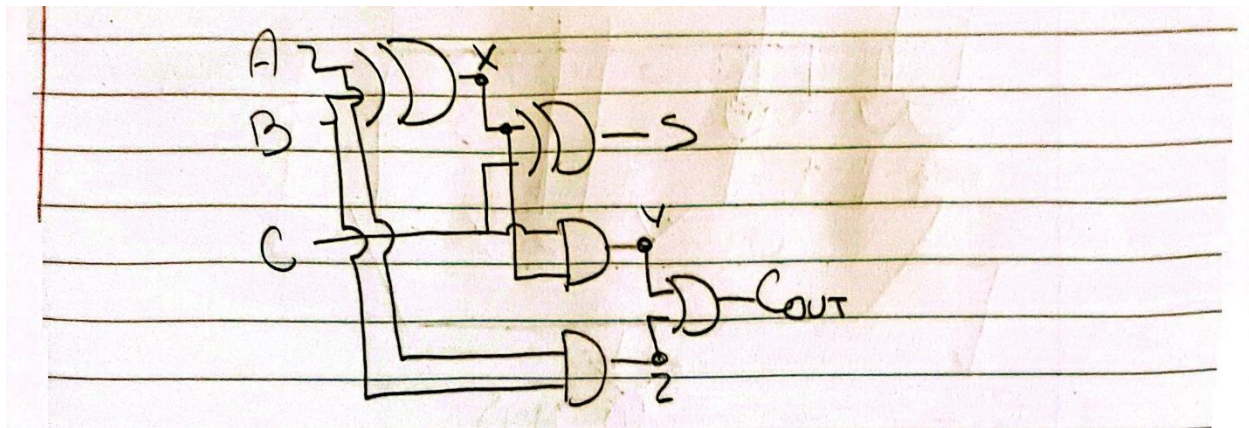


Figure 9: Complete Full Adder Circuit

B. Structural VHDL Implementation

The design of the full adder is stored in full_adder.vhd.

C. N-Bit Ripple Carry Adder

Using the principles from part 4 of generics paired with individual gate definitions and the clock dependance property taken from the TPU design, I designed an N-bit ripple carry adder that switched on rising clock edges. This design is stored in Full_Adder_N.vhd.

D. Testbench and Simulation

I made the testbench by taking inspiration from the TPU element testbench and onescomp testbench. The results of this testbench simulation are shown in figure 10.

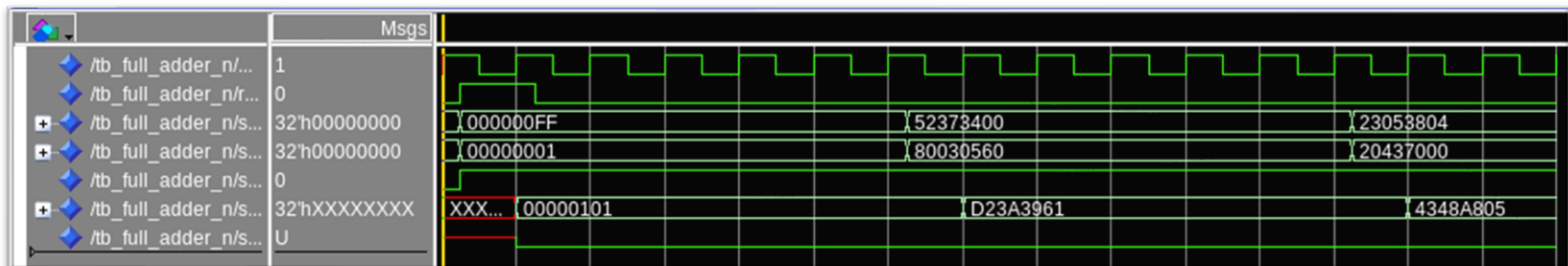


Figure 10: tb_Full_Adder_N waveform output.

In the simulation output, the top two numbers were added with an input carry of 1 to produce the output. All the additions were verified with an online calculator, so it functioned properly.

Problem 7. Adder Subtractor Module

A. Schematic

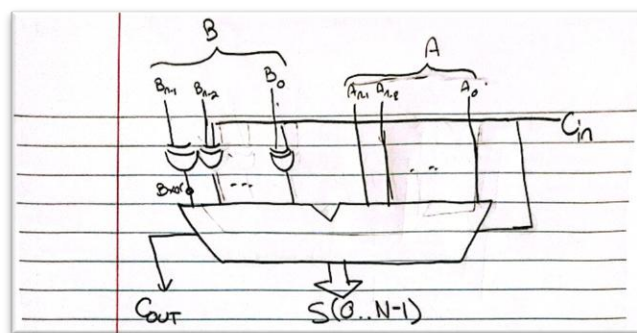


Figure 11: Adder Subtractor Schematic

In this adder subtractor circuit, the C_{in} bit from before now also controls the functionality of the circuit. If it's a 1, the B input gets flipped and added with a 1 to implement a two's complement data structure. Since XOR gates can only have one input be high, when the C_{in} input is high, they essentially become inverters.

B. Structural VHDL implementation

Since the design implements a Full Adder with some additional gates, I built it up from the previous design with a generic statement to generate all the XOR gates for the inverted B input into the adder. This design is saved in add_sub_N.vhd. The testbench is saved in tb_add_sub_N.vhd.

C. Simulation and Testing

For the testbench, I implemented three test cases: a simple FF minus 1, then two very complex numbers to fully test the system. The screenshot of the output is shown in figure 12. I verified each output with an online calculator, so I can confidently say the design works properly.

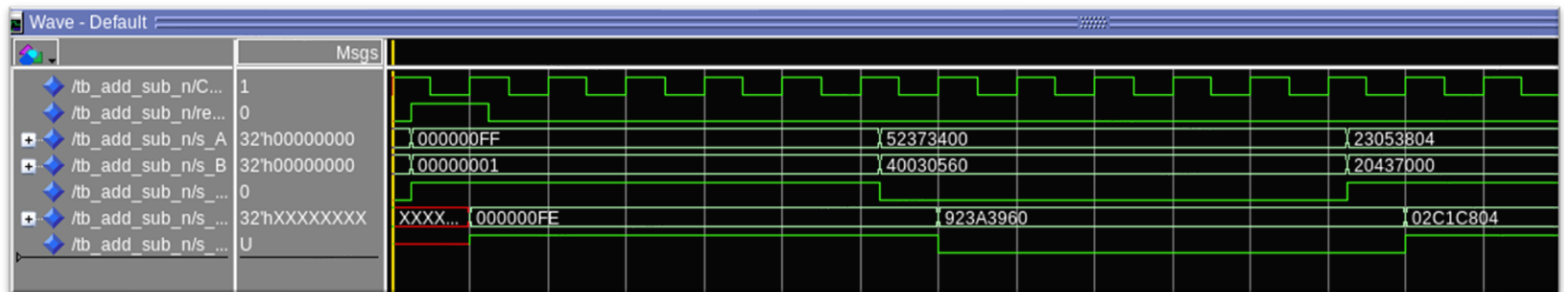


Figure 12: Adder Subtractor testbench simulation output